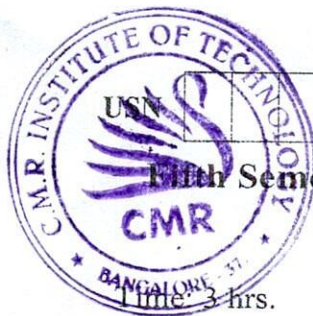




Fifth Semester B.E./B.Tech. Degree Examination, Dec.2024/Jan.2025

Theory of Computation

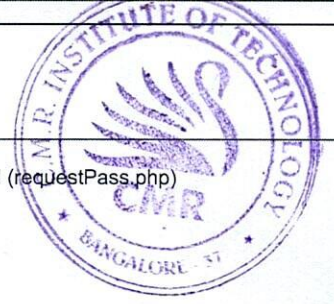
Max. Marks: 100

- Note: 1. Answer any FIVE full questions, choosing ONE full question from each module.
2. M : Marks , L: Bloom's level , C: Course outcomes.

Module – 1				M	L	C														
Q.1	a.	Define the following with example : i) Language ii) String iii) Power of an alphabet.	3	L1	CO1															
	b.	Define DFA. Draw a DFA to accepts. i) The set of all strings that contain a substring aba. ii) To accept the strings of a's and b's that contain not more than three b's. iii) $L = \{w \in \{a, b\}^* : \text{No 2 consecutive characters are same in } w\}$.	10	L3	CO1															
	c.	Convert the following NFA to DFA. <table><tr><td></td><td>0</td><td>1</td></tr><tr><td>$\rightarrow$ p</td><td>{p, q}</td><td>{p}</td></tr><tr><td>q</td><td>{r}</td><td>{r}</td></tr><tr><td>r</td><td>{s}</td><td>ϕ</td></tr><tr><td>* s</td><td>{s}</td><td>{s}</td></tr></table>		0	1	$\rightarrow$ p	{p, q}	{p}	q	{r}	{r}	r	{s}	ϕ	* s	{s}	{s}	7	L2	CO1
	0	1																		
$\rightarrow$ p	{p, q}	{p}																		
q	{r}	{r}																		
r	{s}	ϕ																		
* s	{s}	{s}																		
OR																				
Q.2	a.	Define the following with example : i) Alphabet ii) Reversal of string iii) Concatenation of Languages.	3	L1	CO1															
	b.	Design a DFA for the Language : $L = \{w \in \{0, 1\}^* : w \text{ is a string divisible by } 5\}$.	7	L3	CO1															
	c.	Define NFA. Obtain an ϵ -NFA which accepts strings consisting of 0 or more a's , followed by 0 or more b's followed by 0 or more C's. Also convert it to DFA.	10	L2	CO1															
Module – 2																				
Q.3	a.	Define Regular expression. Write the regular expression for the following languages : i) Strings of a's and b's starting with a and ending with b. ii) Set of strings that consists of alternating 0's and 1's. iii) $L = \{a^n b^m, (n + m) \text{ is even}\}$. iv) $L = \{w : w \text{ mod } 3 = 0, \text{ where } w \in \{a, b\}^*\}$.	10	L2	CO2															

	b.	Minimize the following finite automata using Table filling algorithm :	10	L2	CO2																											
		<table border="1"> <tr> <td>δ</td><td>a</td><td>b</td></tr> <tr> <td>$\rightarrow$ A</td><td>B</td><td>A</td></tr> <tr> <td>B</td><td>A</td><td>C</td></tr> <tr> <td>C</td><td>D</td><td>B</td></tr> <tr> <td>*</td><td>D</td><td>A</td></tr> <tr> <td>E</td><td>D</td><td>F</td></tr> <tr> <td>F</td><td>G</td><td>E</td></tr> <tr> <td>G</td><td>F</td><td>G</td></tr> <tr> <td>H</td><td>G</td><td>D</td></tr> </table>	δ	a	b	$\rightarrow$ A	B	A	B	A	C	C	D	B	*	D	A	E	D	F	F	G	E	G	F	G	H	G	D			
δ	a	b																														
$\rightarrow$ A	B	A																														
B	A	C																														
C	D	B																														
*	D	A																														
E	D	F																														
F	G	E																														
G	F	G																														
H	G	D																														
OR																																
Q.4	a.	Construct ε - NFA for the following Regular expression : i) $(0+1)01(1+0)$ ii) $1(0+1)^*0$ iii) $(0+1)^*011^*$	6	L1	CO2																											
	b.	Obtain the Regular expression that denotes the language accepted by Fig. Q4(b).	6	L3	CO2																											
		Fig. Q4(b) Using Kleene's theorem.																														
	c.	State the Pumping Lemma for the Regular Languages. And also prove that the following languages are not regular. i) $L = \{0^n 1^m \mid n \leq m\}$ ii) $L = \{0^n 1^m 2^n \mid n, m \geq 1\}$.	8	L1	CO2																											
Module - 3																																
Q.5	a.	Design CFG for the following languages : i) $L = \{a^n b^{n+3}, n \geq 0\}$ ii) $L = \{a^i b^j c^k, j = i + k, i \geq 0, k \geq 0\}$ iii) $L = \{w \mid w \bmod 3 > 0 \text{ where } w \in \{a\}^*\}$ iv) $L = \{a^m b^n \mid m \neq n\}$ v) Palindromes over 0 and 1.	10	L3	CO3																											
	b.	Consider the grammar G with productions. $S \rightarrow A b B / A / B$; $A \rightarrow aA / \varepsilon$; $B \rightarrow aB / bB / \varepsilon$. Obtain LMD, RMD and parse tree for the string aabab. Is the given grammar ambiguous?	10	L2	CO3																											
OR																																
Q.6	a.	Define the following with example : i) Context free grammar ii) Left most Derivation iii) Parse tree iv) Ambiguous grammar.	4	L1	CO3																											
	b.	Design PDA for the language : $L = \{a^i b^j c^k \mid i + k = j, i \geq 0, k \geq 0\}$ and show the moves made by the PDA for the string aabbbc.	10	L3	CO3																											

	c.	Convert the following CFG's to PDA : $S \rightarrow aA$; $A \rightarrow aABC / bB / a$; $B \rightarrow b$; $C \rightarrow c$.	6	L2	CO3
Module – 4					
Q.7	a.	Define CNF. Convert the following CFG to CNF. $E \rightarrow E + T / T$ $T \rightarrow T * F / F$ $F \rightarrow (E) / I$ $I \rightarrow Ia / Ib / a / b$.	10	L2	CO4
	b.	Show that $L = \{0^n 1^n 2^n / n \geq 1\}$ is not context free.	4	L2	CO4
	c.	Prove that the family of context free languages is closed under union and concatenation.	6	L1	CO4
OR					
Q.8	a.	Define Greibach Normal Form. Convert the following CFG to GNF. $S \rightarrow AB$; $A \rightarrow aA / bB / b$; $B \rightarrow b$.	6	L2	CO4
	b.	Consider the following CFG : $S \rightarrow ABC / BaB$ $A \rightarrow aA / BaC / aaa$ $B \rightarrow bBb / a / D$ $C \rightarrow CA / AC$ $D \rightarrow \epsilon$ i) What are useless symbols? ii) Eliminate ϵ - productions , Unit productions and useless symbols from the grammar.	10	L3	CO4
	c.	Prove that the following languages are not context free. i) $L = \{a^i / i \text{ is prime}\}$ ii) $L = \{a^{n^2} / n \geq 1\}$.	4	L2	CO3
Module – 5					
Q.9	a.	Define a Turing machine and explain with neat diagram, the working of a basic Turing machine.	6	L1	CO4
	b.	Design a Turing machine to accept the language, $L = \{a^n b^n c^n / n \geq 1\}$. Draw the transition diagram and show the moves for the string aabbcc.	14	L4	CO4
OR					
Q.10	a.	Design a Turing machine to accept palindrome over $\{a, b\}$ and draw the transition diagram.	12	L4	CO5
	b.	Write a short notes on : i) Recursively Enumerable Language. ii) Multitape Turing Machine.	8	L1	CO5



Current Session : 02:00:00

02:00:00 Session is in progress.

[Dashboard \(dashboard.php\)](#)

[Download Question Papers \(downloadQP.php\)](#)

[Request Password \(requestPass.php\)](#)

[Request Question Paper \(qpRequest.php\)](#)

[Discrepancy Tab \(feedback.php\)](#)

C.M.R. INSTITUTE OF TECHNOLOGY KUNDAL HALLI

[LOGOUT ! \(logout.php\)](#)

Feedback/Remarks

Date:17-01-2025 | Session:02:00:00

For Subject Code : BCS503

1. Q. 1b. There to be read as **three**
2. Q. 3a. bm to **b^m**
3. Q. 4c. note to be read as **not**
4. Q. 5c. palinderomes to be read as **palindromes**
5. Q. 7b. 2n to **2ⁿ**
6. Q. 8c. ai to **aⁱ**

CMRIT LIBRARY
BANGALORE - 560 037

1 a) Define the following

Strings

A string is a *finite sequence of symbols selected from some alphabet*. It is generally denoted as w . For example for alphabet $\Sigma = \{0, 1\}$ $w = 010101$ is a string.

Length of a string is denoted as $|w|$ and is defined as the number of positions for the symbol in the string. For the above example length is 6.

Languages

A language is a *set of string all of which are chosen from some Σ^* , where Σ is a particular alphabet*. This means that language L is subset of Σ^* . An example is English language, where the collection of legal English words is a set of strings over the alphabet that consists of all the letters. Another example is the C programming language where the alphabet is a subset of the ASCII characters and programs are subset of strings that can be formed from this alphabet.

Power of an alphabet

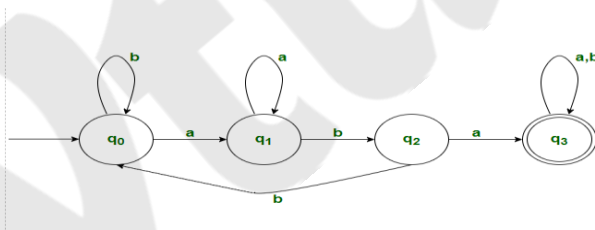
If Σ is an alphabet, the set of all strings can be expressed as a certain length from that alphabet by using exponential notation. The power of an alphabet is denoted by Σ^k and is the set of strings of length k .

For example,

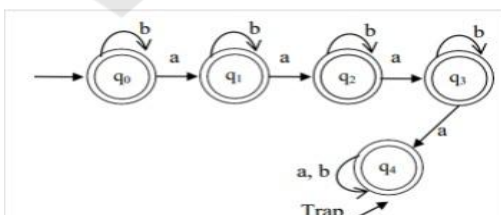
- $\Sigma = \{0, 1\}$
- $\Sigma^1 = \{0, 1\}$ ($2^1=2$)
- $\Sigma^2 = \{00, 01, 10, 11\}$ ($2^2=4$)

1b) Define DFA . draw DFA to accept

i) The set of all strings that contain substring aba

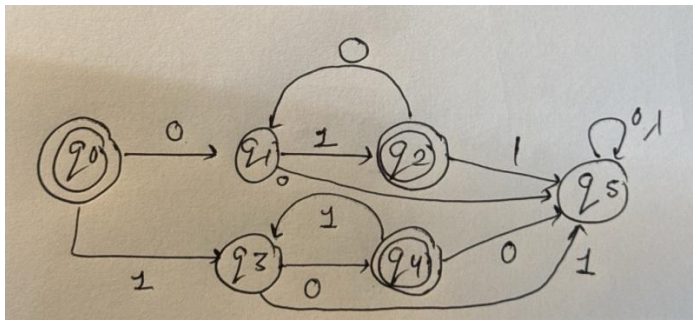


ii) To accept the strings of a's and b's that contain not more than three b's



Not more than 3 b's

iii) $L = \{w \text{ belongs to } \{a,b\}^* : \text{No 2 consecutive characters are same in } w\}$



c) Convert the following NFA to DFA

	0	1
$\rightarrow P$	$\{p,q\}$	$\{p\}$
Q	$\{r\}$	$\{r\}$
R	$\{s\}$	$\emptyset$
$*S$	$\{s\}$	$\{s\}$

Solution :

	0	1
$\rightarrow P$	$\{p,q\}$	$\{p\}$
$\{p,q\}$	$\{p,q,r\}$	$\{p,r\}$
$\{p,q,r\}$	$\{p,q,r,s\}$	$\{p,r\}$
$\{p,r\}$	$\{p,q,s\}$	$\{p\}$
$\{p,q,r,s\}$	$\{p,q,r,s\}$	$\{p,r,s\}$
$\{p,q,s\}$	$\{p,q,r,s\}$	$\{p,r,s\}$
$\{p,r,s\}$	$\{p,q,s\}$	$\{p,s\}$
$\{p,s\}$	$\{p,q,s\}$	$\{p,s\}$

2.a Define the following with example

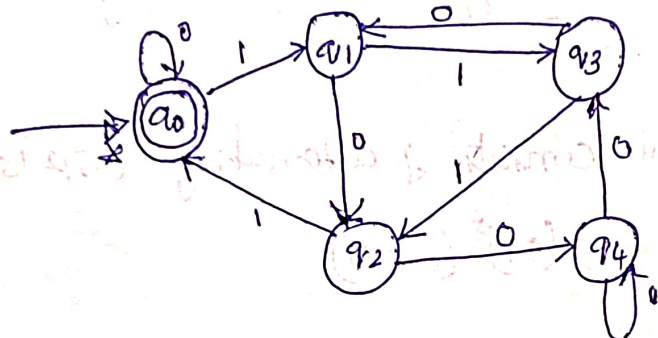
i) Alphabet , ii) Reversal of String, iii) Concatenation of the language

Definition of the above terms

2_b $L = \{w \in \{0,1\}^* : w \text{ is a string divisible by } 5\}$

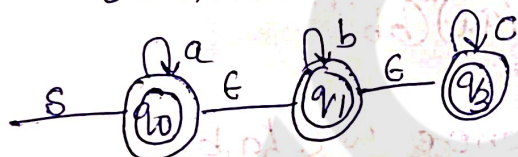
$$= \{\epsilon, 5, 10, 15, \dots\}$$

$$= \{\epsilon, 101, 0101, 1010, \dots\}$$



2_c NFA = $NF_m = (Q, \Sigma, \delta, q_0, F)$

$$\delta: Q \times \Sigma \rightarrow 2^Q$$



$$\delta(q_0, \epsilon) = \text{closure}(q_0) = \{q_0, q_1, q_2\}$$

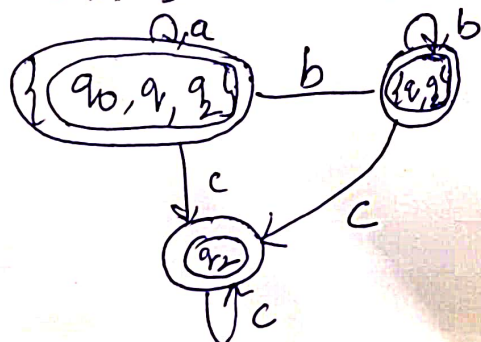
$$\delta(q_1, \epsilon) = \text{closure}(q_1) = \{q_1, q_2\}$$

$$\delta(q_2, \epsilon) = \text{closure}(q_2) = \{q_2\}$$

$$\delta(q_0, q_1, q_2, a) = \{q_0\} \cup \{q_1\} \cup \{q_2\} = \{q_0, q_1, q_2\}$$

$$\delta(q_0, q_2, b) = \{q_1\} = \{q_1, q_2\}$$

$$\delta(q_2, c) = \{q_2\} = \{q_2\}$$



Define Regular expression. write regular expression for the following languages.

Def'n of Regular expression. — o2 M.

(i) strings of a's & b's starting with a & ending with b.

$$a(a+b)^*b$$

(ii) set of strings that consists of alternating 0's & 1's.

$$(0+1+0)^* (0+1)(01)^* (0+1)$$

(iii) $L = \{ a^n b^m, (n+m) \text{ is even} \}$

$$L = \{ \epsilon, abbb, aabb, aaab, \dots \}$$

$$R = a(aa)^*b(bb)^* + (aa)^*(bb)^*$$

(iv) $L = \{ w : |w| \bmod 3 = 0, \text{ where } w \in \{a,b\}^* \}$.

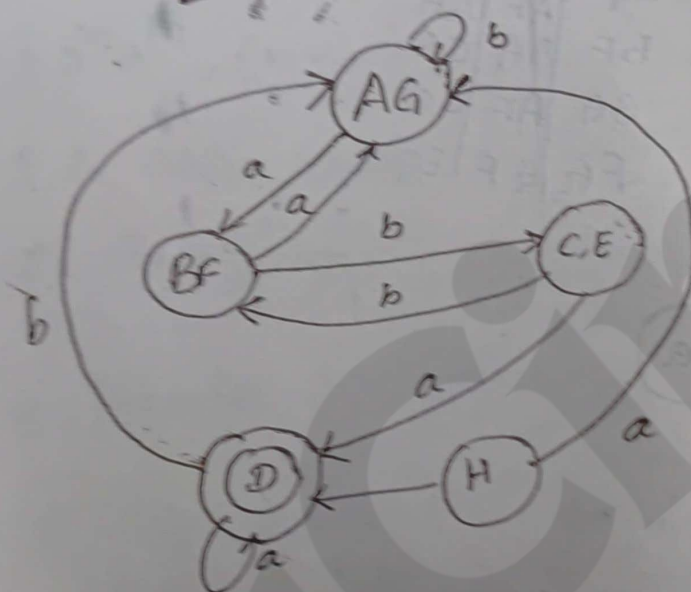
$$L = \{ \epsilon, aaa, aaaaaa, \dots \}$$

$$R = (aaa)^*((a+b)(a+b)(a+b))^*$$

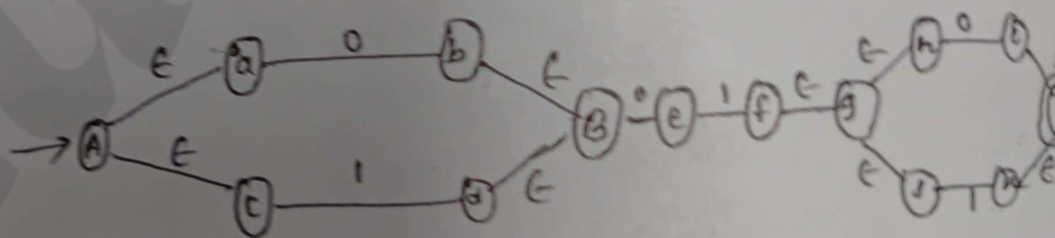
Minimize the following Finite Automata using Table filling algorithm.

δ	a	b
A	B	A
B	A	C
C	D	B
D	D	A
E	D	F
F	G	E
G	F	G

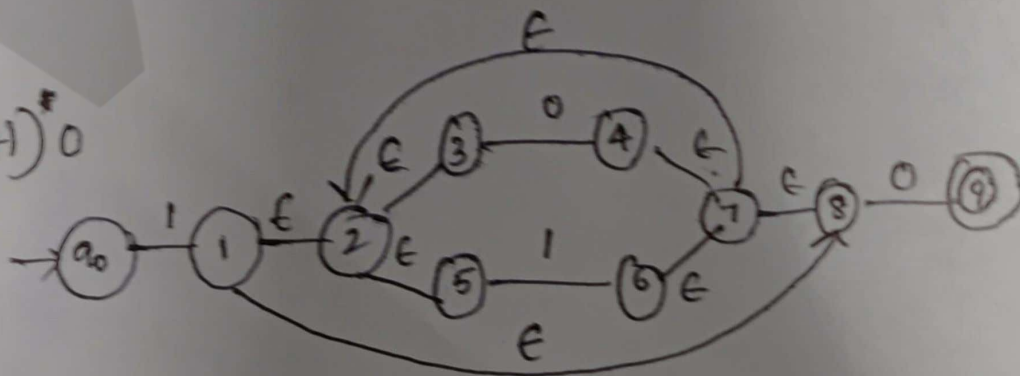
B							
C							
D	X	X	X				
E							
F					X		
G					X		
H					X		
	A	B	C	D	E	F	G



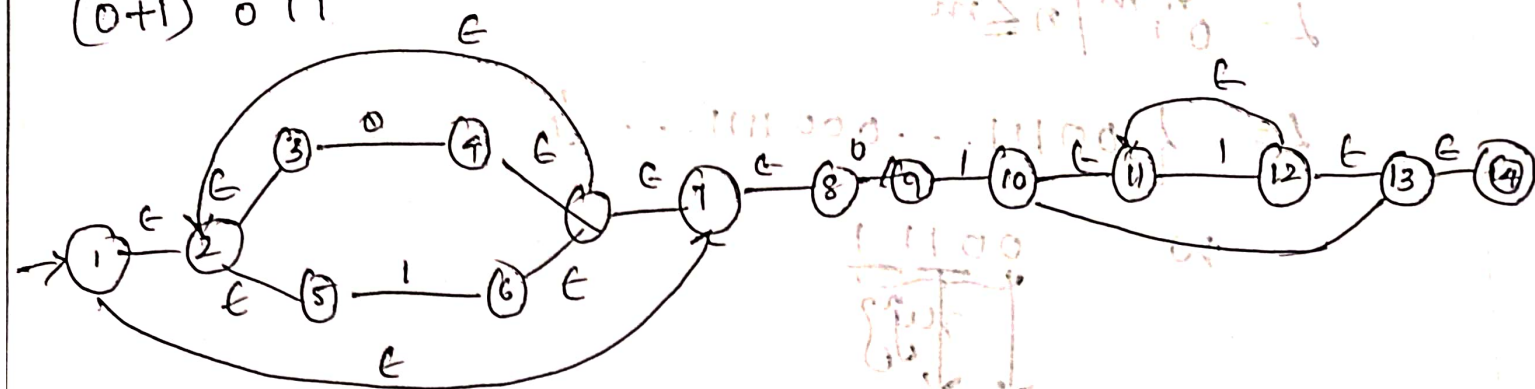
$(0+1)0^*(1+0)^*$



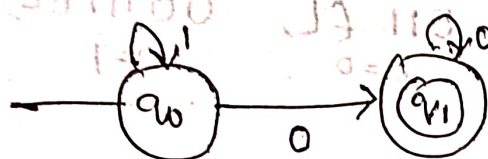
$1(0+1)^*0$



$(0+1)^* 0 11^*$



Regular expression that denotes language accepted



R.E	k=0	k=1	k=2
$R_{11}^{(k)}$	$\epsilon + 1$	1^k	
$R_{12}^{(k)}$	0	$1^* 0$	$1^* 0 (0+1)^*$
$R_{21}^{(k)}$	$\emptyset$	$\emptyset$	
$R_{22}^{(k)}$	$\epsilon + 0 + 1$	$\epsilon + 0 + 1$	

Sol: $1^* 0 (0+1)^*$

State pumping lemma for regular languages and prove that languages are not regular.

$$L = \{0^n 1^m \mid n \leq m\} \quad (ii) \quad L = \{0^n 1^m 2^n \mid n, m \geq 1\}$$

Sol: $xy^n z \in L, \quad |y| \geq 1, \quad \forall n \geq 1$
 $|xz| \leq n, \quad |y| \geq n$

$$(xy^n z) \in L, \quad \forall n = 0, 1, 2, \dots, \infty$$

$$L = 0^n 1^m \mid n \leq m$$

$$L = \{ 00111 \dots 0001111 \dots \}$$

$$w = \begin{array}{cccc} 0 & 0 & 1 & 1 \\ \hline & x & y & z \end{array}$$

$$x=0, y=0, z=1$$

$$0(0)^n 11 = 011 \in L, n=0$$

$$00111 \in L, n=1$$

$$0010111 \notin L$$

$$L = \{ 0^n 1^m 2^n \mid n, m \geq 1 \}$$

$$L = \{ 012, 001122, 000111222 \dots \}$$

$$w = \begin{array}{ccc} 00 & 11 & 22 \\ \hline x & y & z \end{array}$$

$$|y| \geq 1 = |x| \geq 1, |xz| \leq n$$

$$00(11)^n 22, \forall n=0, 1, 2, \dots$$

$$0022 \notin L, \text{ hence they are not regular.}$$

5
a.

$$L = \{ a^n b^{n+3}, n \geq 0 \}$$

$$p \rightarrow s \rightarrow asb \mid bbb, \theta = \{ \epsilon sb, \epsilon a, bb, \epsilon s \}$$

$$\text{Value of } b = bbb, \forall n=0$$

$$abbbb, \forall n=1$$

$$\rightarrow \text{No } a \text{ b's more than the no } a's.$$

ii) $L = \{a^i b^j c^k, j = i+k, i \geq 0, k \geq 0\}$

Sol: $P = S_1 S_2$ $S_1 \rightarrow a S_1 b / \epsilon$, $S_2 \rightarrow a S_2 b / \epsilon$

$G = (\{S_1, S_2, S_3\}, \{a, b, c\}, P, S_1)$

(iii) $L = \{w | w \text{ mod } 3 = 0, \text{ where } w \in a^*\}$

$P = \{S \rightarrow a a a S / a | a a\}$

$G = (\{S\}, \{a\}, P, S)$

(iv) $L = \{a^m b^n | m \neq n\}$

$L = \{a^m b^n | m < n\} \cup \{a^m b^n | m > n\}$

$P: S \rightarrow a S b / A / B, A \rightarrow a A / a, B \rightarrow b B / b / \epsilon$

$G = (\{S, A, B\}, \{a, b\}, P, S)$

(v) palindromes over $\{0, 1\}$

$P: S \rightarrow 0 S 0 / 1 S 1 / \epsilon$

$G = (\{S\}, \{0, 1\}, P, S)$



Consider the grammar G with productions

$S \rightarrow AB|A|B$ $A \rightarrow aA|\epsilon$ $B \rightarrow aB|bB|\epsilon$

Obtain LMD, RMD & parse tree for string

$aaabab$, is grammar is ambiguous.

$S \Rightarrow AB$

$S \Rightarrow AAB$

$S \Rightarrow aaAB$

$S \Rightarrow aaaAB$

$S \Rightarrow aaabaB$

$S \Rightarrow aaababB$

$S \Rightarrow \underline{aaabab}$

$S \Rightarrow B$

$\Rightarrow aB$

$\Rightarrow aaB$

$\Rightarrow aaaB$

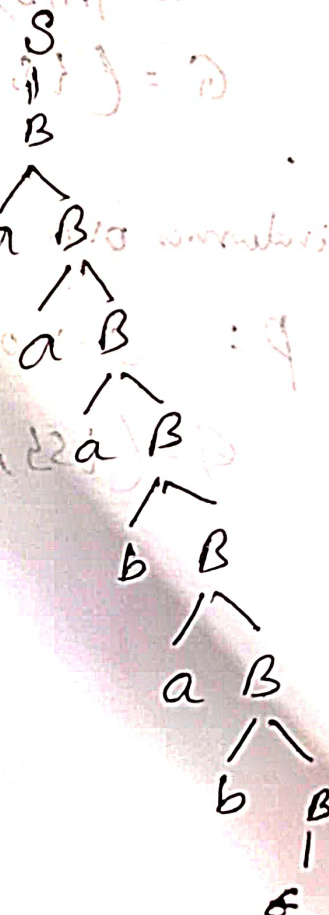
$\Rightarrow aaabaB$

$\Rightarrow aaababB$

$\Rightarrow \underline{aaabab}$

$\Rightarrow \underline{aaabab}$

hence grammar is ambiguous.



ANS 6. A)

The definitions of each of these terms in the context of the **Theory of Automata**:

1. Context-Free Grammar (CFG):

A **Context-Free Grammar (CFG)** is a formal grammar where the left-hand side of every production rule consists of a single non-terminal symbol. It is used to define the syntax of programming languages and can generate context-free languages, which are a subset of formal languages.

A CFG is defined by the 4-tuple $G = (V, \Sigma, R, S)$, where:

- V is a set of variables (non-terminal symbols),
- Σ is a set of terminal symbols (which appear in the strings generated by the grammar),
- R is a set of production rules (each rule is of the form $A \rightarrow \gamma$, where A is a non-terminal and γ is a string of terminals and/or non-terminals),
- S is the start symbol (a specific non-terminal from V).

Example:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

2. Left-Most Derivation:

A **left-most derivation** is a process of deriving a string from the start symbol in a way that, at each step, the left-most non-terminal is always replaced first using a production rule.

In a **left-most derivation**:

- The first non-terminal symbol (from the left) in the string is always chosen for expansion.
- This process continues until all the non-terminal symbols are replaced by terminal symbols.

Example of Left-Most Derivation: Given the CFG:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

To derive ab, the left-most derivation would be:

1. Start with S.
2. Replace S with A B (since $S \rightarrow A B$).
3. Replace A with a (since $A \rightarrow a$).
4. Replace B with b (since $B \rightarrow b$).

Resulting string: ab.

3. Parse Tree:

A **parse tree** (or derivation tree) is a tree that represents the syntactic structure of a string according to a context-free grammar. It shows how the start symbol is expanded into a string by applying production rules. Each internal node represents a non-terminal symbol, and the leaves of the tree correspond to the terminal symbols.

In a **parse tree**:

- The root is the start symbol.
- The internal nodes represent the non-terminals, and the edges represent the application of production rules.
- The leaves represent the terminal symbols, which form the string derived.

Example of Parse Tree: For the derivation of ab using the CFG:

$S \rightarrow A B$

$A \rightarrow a$

$B \rightarrow b$

The parse tree would look like this:

```

  S
 / \
A   B
/   \
a    b
```

4. Ambiguous Grammar:

A **grammar is ambiguous** if there is more than one distinct parse tree or left-most derivation for the same string. In other words, a string can be derived in multiple ways using the same grammar.

An **ambiguous grammar** may lead to confusion or difficulties in interpreting strings because the meaning or structure of the string could vary depending on the derivation or parse tree chosen.

Example of Ambiguous Grammar: Consider the grammar:

$$S \rightarrow S + S \mid S * S \mid a$$

For the string $a + a * a$, there are two possible parse trees (showing different interpretations of the operations):

1. **First parse tree** (associating $*$ first):

```

S
/|\
S+S
/  \
a  S*
    / \
    a  a
```

2. **Second parse tree** (associating $+$ first):

```

S
/|\
S*S
/  \
a  a a
```

Since there are multiple parse trees for the same string, the grammar is **ambiguous**.

Summary:

- **Context-Free Grammar (CFG):** A formal grammar with rules where each production has a single non-terminal on the left-hand side.

- **Left-Most Derivation:** A derivation where, at each step, the left-most non-terminal is replaced first.
- **Parse Tree:** A tree that represents the syntactic structure of a string derived from a CFG.
- **Ambiguous Grammar:** A grammar that can generate a string in more than one way, leading to multiple parse trees.

ANS 6. B)

Language Description:

The designing a Pushdown Automaton (PDA) for the language $L = \{a^i b^j c^k \mid i + k = j\}$. In other words, the number of bb's should be equal to the sum of the number of aa's and cc's in the string. Specifically, for any string of the form $a^i b^j c^k$, the relation $i+k=j$ must hold.

PDA Design:

To design the PDA, we need to make use of a stack to keep track of the balance between aa's and cc's, since their combined count needs to match the count of bb's. Here's how we can approach this:

States:

- q_0 : Initial state.
- q_1 : Reading aa's and pushing AA onto the stack.
- q_2 : Reading bb's and matching with the stack.
- q_3 : Reading cc's and popping from the stack.
- q_f : Accepting state (final state).

Stack Alphabet:

- Z_0 : Bottom-of-stack marker.
- AA: Marker for each aa (each aa increases the count of $i+k$).
- BB: Marker for each bb (matches $i+k$).

Transition Function:

- $\delta(q_0, a, Z_0) = (q_1, AZ_0)$: On reading aa, push AA onto the stack.

- $\delta(q_1, a, A) = (q_1, AA) \setminus \delta(q_1, a, A) = (q_1, A A)$: Continue pushing AA for each aa.
- $\delta(q_1, b, A) = (q_2, \epsilon) \setminus \delta(q_1, b, A) = (q_2, \epsilon)$: On reading bb, pop AA from the stack.
- $\delta(q_2, b, A) = (q_2, \epsilon) \setminus \delta(q_2, b, A) = (q_2, \epsilon)$: Continue popping AA for each bb.
- $\delta(q_2, c, Z_0) = (q_3, Z_0) \setminus \delta(q_2, c, Z_0) = (q_3, Z_0)$: On reading cc, move to state q_3 without changing the stack (since the count of bb's must balance the sum of aa's and cc's).
- $\delta(q_3, c, A) = (q_3, \epsilon) \setminus \delta(q_3, c, A) = (q_3, \epsilon)$: On reading cc, pop AA from the stack.
- $\delta(q_3, \epsilon, Z_0) = (q_f, Z_0) \setminus \delta(q_3, \epsilon, Z_0) = (q_f, Z_0)$: On reading the empty string and reaching the bottom-of-stack marker, accept.

PDA Moves for the String "aabbbc":

Step-by-Step Execution:

Let's walk through how the PDA would process the string "aabbbc" with this design:

1. Start at q_0 , with empty stack Z_0 :

- Read the first symbol aa:
 - $\delta(q_0, a, Z_0) = (q_1, AZ_0) \setminus \delta(q_0, a, Z_0) = (q_1, A Z_0) \rightarrow \text{Stack: } [A, Z_0]$
- Transition to state q_1 .

2. In state q_1 , stack: $[A, Z_0]$:

- Read the second symbol aa:
 - $\delta(q_1, a, A) = (q_1, AA) \setminus \delta(q_1, a, A) = (q_1, A A) \rightarrow \text{Stack: } [A, A, Z_0]$
- Stay in state q_1 .

3. In state q_1 , stack: $[A, A, Z_0]$:

- Read the first symbol bb:
 - $\delta(q_1, b, A) = (q_2, \epsilon) \setminus \delta(q_1, b, A) = (q_2, \epsilon) \rightarrow \text{Stack: } [A, Z_0]$
- Transition to state q_2 .

4. In state q_2 , stack: $[A, Z_0]$:

- Read the second symbol bb:
 - $\delta(q_2, b, A) = (q_2, \epsilon) \setminus \delta(q_2, b, A) = (q_2, \epsilon) \rightarrow \text{Stack: } [Z_0]$

- Stay in state q_2 .
- 5. **In state q_2 , stack: $[Z_0][Z_0]$:**
 - Read the third symbol bb :
 - $\delta(q_2, b, Z_0) = (q_3, Z_0) \setminus \delta(q_2, b, Z_0) = (q_3, Z_0) \rightarrow \text{Stack: } [Z_0][Z_0]$
 - Transition to state q_3 .
- 6. **In state q_3 , stack: $[Z_0][Z_0]$:**
 - Read the first symbol cc :
 - $\delta(q_3, c, Z_0) = (q_3, Z_0) \setminus \delta(q_3, c, Z_0) = (q_3, Z_0) \rightarrow \text{Stack: } [Z_0][Z_0]$
 - Stay in state q_3 .
- 7. **In state q_3 , stack: $[Z_0][Z_0]$:**
 - No more symbols to read, and we reach the bottom-of-stack marker Z_0 :
 - $\delta(q_3, \epsilon, Z_0) = (q_f, Z_0) \setminus \delta(q_3, \epsilon, Z_0) = (q_f, Z_0) \rightarrow \text{Accept.}$

ANS 6 C)

Let's break down the task of converting the given **Context-Free Grammar (CFG)** into a **Pushdown Automaton (PDA)**.

Given Grammar:

- $S \rightarrow aAS \setminus \text{to } aA$
- $A \rightarrow aABC \setminus bB \setminus aA \setminus \text{to } aABC \setminus \text{mid } bB \setminus \text{mid } a$
- $B \rightarrow bB \setminus \text{to } b$
- $C \rightarrow cC \setminus \text{to } c$

Steps for Conversion:

To convert a CFG into a PDA, we'll follow the standard procedure of constructing a PDA that accepts the language generated by the CFG. The PDA will operate by pushing symbols onto the stack as it processes the input and popping them when necessary according to the production rules of the CFG.

PDA Design:

1. **States:**

- q_0 : The initial state.
- q_1 : A working state for expanding non-terminals using the production rules.
- q_f : The accepting (final) state.

2. Alphabet:

- The **input alphabet** is $\{a, b, c\}$.
- The **stack alphabet** consists of the non-terminals and terminal symbols from the grammar: $\{S, A, B, C, a, b, c\}$.

3. Stack Operation:

- The PDA will push symbols onto the stack for non-terminal symbols and pop them when processing the corresponding terminal symbols.

4. Start State:

- The PDA will start in q_0 and begin with the start symbol S on the stack.

5. Transitions: The transitions will be defined to mimic the production rules of the grammar. Let's define these transitions based on the productions.

Conversion Process (Transition Function):

1. Start with the start symbol S on the stack:

- Transition from the start state q_0 to process the start symbol S .
- $\delta(q_0, \epsilon, Z_0) = (q_1, SZ_0)$
 - Here, Z_0 is the bottom-of-stack marker, and we push S onto the stack.

2. Handle $S \rightarrow aAS$ to aA :

- From state q_1 , when reading a , we replace S with aA , pushing A onto the stack after reading the a .
- $\delta(q_1, a, S) = (q_1, A)$
 - This simulates the production $S \rightarrow aAS$ to aA .

3. Handle $A \rightarrow aABCA$ to $aABC$:

- When the top of the stack is AA , we can expand it into $aABCaABC$. This requires pushing $ABCABC$ onto the stack and consuming the terminal aa .

- $\delta(q_1, a, A) = (q_1, ABC) \setminus \delta(q_1, a, A) = (q_1, A B C)$

4. Handle $A \rightarrow bBA$ to bB :

- If we encounter a bb while the top of the stack is AA , we replace AA with $bBbB$, consuming the terminal bb .
- $\delta(q_1, b, A) = (q_1, B) \setminus \delta(q_1, b, A) = (q_1, B)$

5. Handle $A \rightarrow aA$ to a :

- If AA is on the top of the stack and we encounter an aa , we pop AA (since $A \rightarrow aA$ to a) and consume the aa .
- $\delta(q_1, a, A) = (q_1, \epsilon) \setminus \delta(q_1, a, A) = (q_1, \epsilon)$

6. Handle $B \rightarrow bB$ to b :

- If BB is on top of the stack, we replace it with the terminal bb , consuming bb .
- $\delta(q_1, b, B) = (q_1, \epsilon) \setminus \delta(q_1, b, B) = (q_1, \epsilon)$

7. Handle $C \rightarrow cC$ to c :

- If CC is on the top of the stack, we replace it with the terminal cc , consuming cc .
- $\delta(q_1, c, C) = (q_1, \epsilon) \setminus \delta(q_1, c, C) = (q_1, \epsilon)$

8. Accepting Condition:

- The PDA reaches an accepting state q_f when the input string has been processed, and the stack is empty (all symbols have been matched and popped).
- $\delta(q_1, \epsilon, Z_0) = (q_f, Z_0) \setminus \delta(q_1, \epsilon, Z_0) = (q_f, Z_0)$

PDA Diagram (Conceptually):

1. Start state q_0 :

- $\delta(q_0, \epsilon, Z_0) = (q_1, SZ_0) \setminus \delta(q_0, \epsilon, Z_0) = (q_1, S Z_0)$

2. State q_1 : (Process rules for SS and AA)

- $\delta(q_1, a, S) = (q_1, A) \setminus \delta(q_1, a, S) = (q_1, A)$ (for $S \rightarrow aAS$ to aA)
- $\delta(q_1, a, A) = (q_1, ABC) \setminus \delta(q_1, a, A) = (q_1, A B C)$ (for $A \rightarrow aABCA$ to $aABC$)
- $\delta(q_1, b, A) = (q_1, B) \setminus \delta(q_1, b, A) = (q_1, B)$ (for $A \rightarrow bBA$ to bB)
- $\delta(q_1, a, A) = (q_1, \epsilon) \setminus \delta(q_1, a, A) = (q_1, \epsilon)$ (for $A \rightarrow aA$ to a)

- $\delta(q_1, b, B) = (q_1, \epsilon) \setminus \delta(q_1, b, B) = (q_1, \epsilon)$ (for $B \rightarrow bB$ \to b)
- $\delta(q_1, c, C) = (q_1, \epsilon) \setminus \delta(q_1, c, C) = (q_1, \epsilon)$ (for $C \rightarrow cC$ \to c)

3. Accepting state q_f :

- $\delta(q_1, \epsilon, Z_0) = (q_f, Z_0) \setminus \delta(q_1, \epsilon, Z_0) = (q_f, Z_0)$

Example of Execution for the String "aaabbbc":

Let's break down how the PDA works on the string "aaabbbc":

1. Start in q_0 with the stack Z_0 .
 - $\delta(q_0, \epsilon, Z_0) = (q_1, SZ_0) \setminus \delta(q_0, \epsilon, Z_0) = (q_1, SZ_0)$
2. In state q_1 , with the stack $[S, Z_0]$:
 - Read aa: $\delta(q_1, a, S) = (q_1, A) \setminus \delta(q_1, a, S) = (q_1, A)$
 - Stack: $[A, Z_0]$
3. In state q_1 , with the stack $[A, Z_0]$:
 - Read aa: $\delta(q_1, a, A) = (q_1, ABC) \setminus \delta(q_1, a, A) = (q_1, ABC)$
 - Stack: $[A, B, C, Z_0]$
4. In state q_1 , with the stack $[A, B, C, Z_0]$:
 - Read aa: $\delta(q_1, a, A) = (q_1, \epsilon) \setminus \delta(q_1, a, A) = (q_1, \epsilon)$
 - Stack: $[B, C, Z_0]$
5. In state q_1 , with the stack $[B, C, Z_0]$:
 - Read bb: $\delta(q_1, b, B) = (q_1, \epsilon) \setminus \delta(q_1, b, B) = (q_1, \epsilon)$
 - Stack: $[C, Z_0]$
6. In state q_1 , with the stack $[C, Z_0]$:
 - Read bb: $\delta(q_1, b, C) = (q_1, \epsilon) \setminus \delta(q_1, b, C) = (q_1, \epsilon)$
 - Stack: $[Z_0]$
7. In state q_1 , with the stack $[Z_0]$:
 - Read cc: $\delta(q_1, c, Z_0) = (q_f, Z_0) \setminus \delta(q_1, c, Z_0) = (q_f, Z_0)$
 - Stack: $[Z_0]$ (reached the accepting state)

MODULE 4

ANS 7. A)

Definition of CNF (Chomsky Normal Form)

A Context-Free Grammar (CFG) is said to be in Chomsky Normal Form (CNF) if all of its production rules satisfy the following conditions:

1. Each production is of the form:
 - $A \rightarrow BC$, where A is a non-terminal and B and C are non-terminals (no terminal symbols appear on the right-hand side of the production).
 - OR
 - $A \rightarrow a$, where a is a terminal symbol (a production that directly leads to a terminal).
2. The start symbol can have the form $S \rightarrow \epsilon$, if the language includes the empty string (which is rare and optional in CNF).

To convert a CFG into CNF, we generally follow these steps:

- Eliminate null (ϵ) productions.
- Eliminate unit productions.
- Eliminate useless symbols.
- Ensure that all productions are of the form $A \rightarrow BC$ or $A \rightarrow a$, by introducing new variables (non-terminals) if needed.

Given Grammar:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid i$
- $i \rightarrow i a \mid i b \mid i$

Step-by-Step Conversion to CNF:

We will convert the given grammar into CNF step-by-step. First, let's simplify the grammar and follow the rules of CNF conversion.

1. Eliminate Null Productions (ϵ -productions):

There are no epsilon productions (productions of the form $A \rightarrow \epsilon$) in the given grammar, so we don't need to eliminate any.

2. Eliminate Unit Productions:

A unit production is a production of the form $A \rightarrow B$, where both A and B are non-terminals. We need to replace unit productions by expanding them with their respective definitions.

Let's look at the productions involving unit productions:

- $E \rightarrow TE$ (from $E \rightarrow E+TE$ and $E \rightarrow TE$) is a unit production.
- $T \rightarrow FT$ (from $T \rightarrow T*FT$ and $T \rightarrow FT$) is another unit production.

We need to replace these unit productions with the corresponding right-hand side.

- Replace $E \rightarrow TE$ with $E \rightarrow T*F|FE$ (expand the production rules for T).
- Replace $T \rightarrow FT$ with $T \rightarrow (E)|IT$ (expand the production rules for F).

After expanding the unit productions, the grammar becomes:

- $E \rightarrow E+T|T*F|FE$
- $T \rightarrow T*F|FT|(E)|IT$
- $F \rightarrow (E)|IF$
- $I \rightarrow Ia|Ib|a|b$

3. Eliminate Left Recursion:

The given grammar contains left recursion, which needs to be removed in order to convert the grammar to CNF.

For example, the rule $E \rightarrow E+TE$ is left-recursive. We eliminate left recursion by rewriting the production rules.

- For $E \rightarrow E+TE$, we introduce a new non-terminal $E'E'$ and rewrite the productions as:
 - $E \rightarrow TE'E'$

- $E' \rightarrow +TE' \mid \epsilon E' \mid \epsilon T E' \mid \epsilon$

Now, the rules for EE become:

- $E \rightarrow TE'E \mid \epsilon T E'$
- $E' \rightarrow +TE' \mid \epsilon E' \mid \epsilon T E' \mid \epsilon$

4. Ensure All Productions Are in CNF:

Now we need to ensure that every production is in the form of $A \rightarrow BCA \mid BC$ or $A \rightarrow aA \mid a$. We may need to introduce new non-terminals to achieve this.

For the production $E' \rightarrow +TE'E \mid \epsilon T E'$:

- We can't have the terminal $++$ in the production like this, so we introduce a new non-terminal X_+X_+ to replace the terminal $++$.
- So, $X_+ \rightarrow +X_+ \mid +$.

Thus, we modify the grammar as follows:

- $E' \rightarrow X_+TE' \mid \epsilon E' \mid \epsilon X_+ T E' \mid \epsilon$
- $X_+ \rightarrow +X_+ \mid +$

For the production $T \rightarrow T*FT \mid \epsilon T * F$:

- We introduce a new non-terminal X_*X_* to replace the terminal $*$.
- So, $X_* \rightarrow *X_* \mid *$.
- Thus, we modify TT to be:

- $T \rightarrow X_*FT' \mid FT \mid X_* F T' \mid F$
 - $T' \rightarrow X_*FT' \mid \epsilon T' \mid \epsilon X_* F T' \mid \epsilon$

For the production $F \rightarrow (E)F \mid (E)$:

- We introduce a new non-terminal $X_()X_()$ to replace the terminal $(($ and $X_()X_()'$ to replace $))$.
- So, $X_() \rightarrow (X_() \mid ($ and $X_() \rightarrow X_()'$ to $)$.
- Modify FF to:
 - $F \rightarrow X_()EX_()'$ if $F \rightarrow X_() E X_()'$ mid I

For the production $I \rightarrow IaI \mid Ia$:

- We introduce a new non-terminal X_a to replace the terminal a .
- $X_a \rightarrow aX_a \mid a$.
- Thus, modify I to:
 - $I \rightarrow IX_a | Xab | Xa \mid I X_a \mid X_a b \mid X_a$

Now, the grammar becomes:

- $E \rightarrow TE'E \mid TE'$
- $E' \rightarrow X+TE' \mid \epsilon E' \mid X_+ T E' \mid \epsilon$
- $X+ \rightarrow +X_+ \mid +$
- $T \rightarrow X*FT' \mid FT \mid X_* F T' \mid F$
- $T' \rightarrow X*FT' \mid \epsilon T' \mid X_* F T' \mid \epsilon$
- $X* \rightarrow *X_* \mid *$
- $F \rightarrow X()EX() \mid IF \mid X_() E X_() \mid I$
- $X() \rightarrow (X_() \mid ($
- $X()' \rightarrow)X_() \mid)$
- $I \rightarrow IX_a | Xab | Xa \mid I X_a \mid X_a b \mid X_a$
- $X_a \rightarrow aX_a \mid a$
- $X_b \rightarrow bX_b \mid b$

Final Grammar in CNF:

The resulting grammar in Chomsky Normal Form (CNF) is as follows:

1. $E \rightarrow TE'E \mid TE'$
2. $E' \rightarrow X+TE' \mid \epsilon E' \mid X_+ T E' \mid \epsilon$
3. $X+ \rightarrow +X_+ \mid +$
4. $T \rightarrow X*FT' \mid FT \mid X_* F T' \mid F$
5. $T' \rightarrow X*FT' \mid \epsilon T' \mid X_* F T' \mid \epsilon$
6. $X* \rightarrow *X_* \mid *$
7. $F \rightarrow X()EX() \mid IF \mid X_() E X_() \mid I$

8. $X() \rightarrow (X_()) \setsto ($
9. $X()' \rightarrow)X_()' \setsto)$
10. $I \rightarrow IXa|Xab|XaI \setsto I X_a \setmid X_a b \setmid X_a$
11. $Xa \rightarrow aX_a \setsto a$
12. $Xb \rightarrow bX_b \setsto b$

ANS 7. B)

The language $L = \{0^n 1^n 2^n | n \geq 1\}$ is not a context-free language (CFL). To show this, we can use the pumping lemma for context-free languages. The pumping lemma is a proof technique that can be used to show that certain languages are not context-free.

Pumping Lemma for Context-Free Languages:

The pumping lemma for CFLs states that for any context-free language L , there exists a constant p (called the pumping length) such that any string w in L with $|w| \geq p$ can be decomposed into five parts, $w = uvxyz$, such that:

1. $|vxy| \leq p$
2. $|v| \geq 1$
3. For all $i \geq 0$, the string $uv^i x y^i z$ is in L .

In other words, we can "pump" the middle part of the string (represented by v and y) any number of times, and the resulting string will still belong to the language.

Proof:

To prove that the language $L = \{0^n 1^n 2^n | n \geq 1\}$ is not context-free, we will use the pumping lemma.

Step 1: Assume that the language is context-free.

Assume, for the sake of contradiction, that L is a context-free language. Then, by the pumping lemma for CFLs, there exists a pumping length p such that any string $w \in L$ with $|w| \geq p$ can be decomposed into five parts $w = uvxyz$, where the conditions of the pumping lemma hold.

Step 2: Choose a string from L .

Let's choose the string $w = 0^p 1^p 2^p \in L$, where p is the pumping length given by the pumping lemma. The string has length $3p$, which is greater than or equal to the pumping length, so it must be pumpable according to the pumping lemma.

Step 3: Decompose the string w .

According to the pumping lemma, we can decompose w as $w = uvxyz$, where:

- $|vxy| \leq p$
- $|vy| \geq 1$
- For all $i \geq 0$, the string $uv^i x y^i z \in L$.

Since $w = 0^p 1^p 2^p$, we know that the string consists of three distinct sections: 0^p , 1^p , and 2^p . The substring vxy must be entirely contained within one or two of these sections, because $|vxy| \leq p$ and each section has length p .

Step 4: Analyze possible locations for vxy .

Let's consider the possible cases for the location of the substring vxy :

1. Case 1: vxy is entirely within the first block (the 0^p block).
 - In this case, v and y will both consist of 0's, and pumping v and y will add more 0's.
 - After pumping, the string will have more 0's than 1's and 2's, which cannot be in the form $0^n 1^n 2^n$. Thus, the string will not belong to L .
2. Case 2: vxy is entirely within the second block (the 1^p block).
 - In this case, v and y will both consist of 1's, and pumping v and y will add more 1's.
 - After pumping, the string will have more 1's than 0's and 2's, which again cannot be in the form $0^n 1^n 2^n$. Thus, the string will not belong to L .
3. Case 3: vxy is entirely within the third block (the 2^p block).
 - In this case, v and y will both consist of 2's, and pumping v and y will add more 2's.
 - After pumping, the string will have more 2's than 0's and 1's, which also cannot be in the form $0^n 1^n 2^n$. Thus, the string will not belong to L .

4. Case 4: $vxyvxy$ spans across two blocks (e.g., between the 0^p0^p and 1^p1^p blocks, or between the 1^p1^p and 2^p2^p blocks).
 - In this case, pumping vv and yy will alter the balance between the different symbols (0's, 1's, and 2's).
 - For example, if $vxyvxy$ spans the 0's and 1's, pumping will result in an unequal number of 0's and 1's, violating the structure of the string, which requires equal numbers of 0's, 1's, and 2's.

Step 5: Conclusion.

In all possible cases, pumping the string results in a string that no longer belongs to L . This contradicts the pumping lemma, which states that for a context-free language, pumping should always produce strings that still belong to the language.

Therefore, our assumption that L is a context-free language must be false. Hence, the language $L = \{0^n 1^n 2^n \mid n \geq 1\}$ is not a context-free language.

Final Answer:

The language $L = \{0^n 1^n 2^n \mid n \geq 1\}$ is not context-free, as shown by applying the pumping lemma for context-free languages.

ANS 7. C)

Proof that Context-Free Languages (CFLs) are Closed under Union and Concatenation

In this proof, we will demonstrate that the class of Context-Free Languages (CFLs) is closed under union and concatenation. This means that if L_1 and L_2 are both CFLs, then:

- The union $L_1 \cup L_2$ is also a CFL.
- The concatenation $L_1 \cdot L_2$ is also a CFL.

We will prove both closure properties separately by constructing context-free grammars (CFGs) for the union and concatenation of two CFLs.

1. Closure under Union

Goal:

If L_1 and L_2 are context-free languages, then $L_1 \cup L_2$ is also a context-free language.

Proof:

Let $G_1 = (V_1, \Sigma, R_1, S_1)$ be a context-free grammar for L_1 , and let $G_2 = (V_2, \Sigma, R_2, S_2)$ be a context-free grammar for L_2 , where:

- V_1 and V_2 are sets of non-terminals,
- Σ is the alphabet,
- R_1 and R_2 are sets of production rules,
- S_1 and S_2 are the start symbols for G_1 and G_2 , respectively.

To construct a grammar for $L_1 \cup L_2$, we create a new start symbol S , and define a new grammar $G = (V, \Sigma, R, S)$, where:

1. Non-terminal set: $V = V_1 \cup V_2 \cup \{S\}$, the union of the non-terminals from both grammars and the new start symbol S .
2. Production rules:
 - $S \rightarrow S_1 S_2$
 - All the rules from G_1 : R_1
 - All the rules from G_2 : R_2
3. Start symbol: S , which can derive either S_1 or S_2 .

Explanation:

- S can derive either the start symbol S_1 from G_1 or the start symbol S_2 from G_2 , allowing the new grammar to generate strings from either L_1 or L_2 .
- The grammar G generates exactly the union of L_1 and L_2 , i.e., $L(G) = L_1 \cup L_2$.

Since this is a valid context-free grammar, it follows that CFLs are closed under union.

2. Closure under Concatenation

Goal:

If L_1 and L_2 are context-free languages, then $L_1 \cdot L_2$ (the concatenation of L_1 and L_2) is also a context-free language.

Proof:

Let $G_1=(V_1,\Sigma,R_1,S_1)$ be a context-free grammar for L_1 , and let $G_2=(V_2,\Sigma,R_2,S_2)$ be a context-free grammar for L_2 .

To construct a grammar for $L_1 \cdot L_2$, we create a new start symbol S and define a new grammar $G=(V,\Sigma,R,S)$, where:

1. Non-terminal set: $V=V_1 \cup V_2 \cup \{S\}$, the union of the non-terminals from both grammars and the new start symbol S .
2. Production rules:
 - $S \rightarrow S_1 S_2$, where S_1 is the start symbol for G_1 and S_2 is the start symbol for G_2 .
 - All the rules from G_1 : R_1
 - All the rules from G_2 : R_2
3. Start symbol: S , which produces the concatenation of the strings generated by L_1 and L_2 (i.e., $S \rightarrow S_1 S_2$).

Explanation:

- The start symbol S in the new grammar produces a string from L_1 followed by a string from L_2 , which is the definition of concatenation.
- The rest of the grammar generates strings from L_1 and L_2 according to the original grammars G_1 and G_2 .

Since this is a valid context-free grammar, it follows that CFLs are closed under concatenation.

Conclusion:

We have shown that context-free languages are closed under both union and concatenation by constructing context-free grammars for these operations. Therefore:

- CFLs are closed under union.
- CFLs are closed under concatenation.

Thus, the class of context-free languages is indeed closed under both the union and concatenation operations.

ANS 8. A)

Greibach Normal Form (GNF)

A Context-Free Grammar (CFG) is said to be in Greibach Normal Form (GNF) if all of its production rules are of the form:

- $A \rightarrow a\alpha A$ to $a\alpha$

Where:

- A is a non-terminal,
- a is a terminal symbol,
- α is a string of non-terminals (which can be empty, i.e., α can be the empty string ϵ).

In GNF, the right-hand side of each production must start with a terminal symbol, followed by zero or more non-terminals.

Steps to Convert a CFG to GNF

The general steps to convert a CFG to Greibach Normal Form are:

1. Eliminate epsilon (ϵ) productions: Remove any production of the form $A \rightarrow \epsilon$, unless ϵ is part of the language.
2. Eliminate unit productions: Remove any production of the form $A \rightarrow B$, where both A and B are non-terminals.
3. Eliminate left recursion: A CFG in GNF should not have left recursion (i.e., $A \rightarrow A\alpha$ type rules). Left recursion must be removed by applying a transformation.
4. Rearrange rules into GNF: Convert all production rules so that they are in the form $A \rightarrow a\alpha A$ to $a\alpha$.

Given Grammar:

- $S \rightarrow ABS$ to AB
- $A \rightarrow aA|bB|bA$ to $aA \mid bB \mid bA$
- $B \rightarrow bB$ to b

Step-by-Step Conversion to GNF

We need to convert the given CFG into Greibach Normal Form.

Step 1: Eliminate epsilon (ϵ) productions

- The grammar does not contain any epsilon productions (i.e., no rule of the form $A \rightarrow \epsilon A$ \to \epsilon A \to \epsilon A).

Step 2: Eliminate unit productions

A unit production is a rule where a non-terminal produces another non-terminal directly, like $A \rightarrow BA$ \to B. We need to eliminate these.

- We have $A \rightarrow bBA$ \to bB. Here, AA produces $bBbB$, so we can expand BB using its production rule $B \rightarrow bB$ \to b.

Thus, $A \rightarrow bBA$ \to bB can be rewritten as $A \rightarrow bbA$ \to bb.

After expanding the unit production, the grammar becomes:

- $S \rightarrow ABS$ \to AB
- $A \rightarrow aA|bb|bA$ \to aA \mid bb \mid b
- $B \rightarrow bB$ \to b

Step 3: Eliminate left recursion

The rule $A \rightarrow aAA$ \to aA is left recursive because AA appears on the right-hand side immediately after the terminal symbol aa . To eliminate the left recursion, we rewrite the rules for AA using the following standard method for removing left recursion:

1. Split the production rules for AA into two parts:
 - $A \rightarrow aA'A$ \to aA'
 - $A' \rightarrow aA'|bb|bA'$ \to aA' \mid bb \mid b

Here, we replace the left-recursive rule with a new non-terminal $A'A'$, and the new production $A'A'$ generates the recursive and non-recursive parts.

Step 4: Rearranging to GNF

Now, all rules need to start with a terminal symbol. The current grammar rules are:

- $S \rightarrow ABS$ \to AB
- $A \rightarrow aA'|bb|bA$ \to aA' \mid bb \mid b
- $A' \rightarrow aA'|bb|bA'$ \to aA' \mid bb \mid b
- $B \rightarrow bB$ \to b

We will now express each rule in GNF, i.e., where each production starts with a terminal.

- $S \rightarrow ABS$ to AB can remain as it is, as AA and BB will be expanded recursively into terminal symbols.
- $A \rightarrow aA'A$ to aA' is in GNF.
- $A \rightarrow bbA$ to bb is in GNF (since it starts with terminal bb).
- $A \rightarrow bA$ to b is also in GNF.
- $A' \rightarrow aA'A'$ to aA' is already in GNF.
- $A' \rightarrow bbA'$ to bb is in GNF.
- $A' \rightarrow bA'$ to b is in GNF.
- $B \rightarrow bB$ to b is already in GNF.

Thus, after converting the CFG, we have the following Greibach Normal Form grammar:

Final Grammar in GNF:

- $S \rightarrow ABS$ to AB
- $A \rightarrow aA'|bb|bA$ to aA' \mid bb \mid b
- $A' \rightarrow aA'|bb|bA'$ to aA' \mid bb \mid b
- $B \rightarrow bB$ to b

AND 8. B)

Let's go through the steps systematically to eliminate null productions, unit productions, and useless symbols from the given Context-Free Grammar (CFG).

Given CFG:

- $S \rightarrow ABC|BaBS$ to ABC \mid BaB
- $A \rightarrow aA|BaC|aaaA$ to aA \mid BaC \mid aaa
- $B \rightarrow bBb|a|DB$ to bBb \mid a \mid D
- $C \rightarrow CA|ACC$ to CA \mid AC
- $D \rightarrow \epsilon D$ to ϵ (Null production)

Step 1: Eliminate Null Productions

A null production is a production of the form $A \rightarrow \epsilon A$ to ϵ , where ϵ represents the empty string. In this grammar, $D \rightarrow \epsilon D$ to ϵ is a null production.

To eliminate null productions, we replace occurrences of non-terminals that could derive ϵ with alternatives that account for the possibility of producing nothing.

- $D \rightarrow \epsilon D$ implies that DD can be removed or replaced by ϵ in any production where it appears.

1.1. Modify productions containing DD :

- $B \rightarrow bBb|a|DB \rightarrow bBb \mid a \mid D$
 - Since $D \rightarrow \epsilon D$, we need to replace DD with ϵ in the production $B \rightarrow DB \rightarrow D$. This results in an additional production: $B \rightarrow \epsilon B \rightarrow \epsilon$.
 - So, $B \rightarrow bBb|a|\epsilon B \rightarrow bBb \mid a \mid \epsilon$.

1.2. Modify productions containing BB :

Since $B \rightarrow \epsilon B \rightarrow \epsilon$, we need to consider the following rules:

- In the rule $S \rightarrow BaBS \rightarrow BaB$, replace BB with ϵ to get:
 - $S \rightarrow BaB|aS \rightarrow BaB \mid a$
 - This means we add a new production: $S \rightarrow aS \rightarrow a$.
- In the rule $A \rightarrow BaCA \rightarrow BaC$, replace BB with ϵ to get:
 - $A \rightarrow aC|aaaA \rightarrow aC \mid aaa$.

After eliminating null productions, the grammar becomes:

- $S \rightarrow ABC|BaB|aS \rightarrow ABC \mid BaB \mid a$
- $A \rightarrow aA|BaC|aaaA \rightarrow aA \mid BaC \mid aaa$
- $B \rightarrow bBb|a|\epsilon B \rightarrow bBb \mid a \mid \epsilon$
- $C \rightarrow CA|ACC \rightarrow CA \mid AC$
- $D \rightarrow \epsilon D \rightarrow \epsilon$

Step 2: Eliminate Unit Productions

A unit production is a production of the form $A \rightarrow BA \rightarrow B$, where AA and BB are non-terminals. Let's eliminate the unit productions:

- $B \rightarrow DB \rightarrow D$ is a unit production. Since $D \rightarrow \epsilon D \rightarrow \epsilon$, we replace $B \rightarrow DB \rightarrow D$ with $B \rightarrow \epsilon B \rightarrow \epsilon$.

- So, we already have the production $B \rightarrow \epsilon B$ to ϵ , so we can ignore the unit production $B \rightarrow DB$ to D .

After eliminating unit productions, the grammar becomes:

- $S \rightarrow ABC | BaB | aS$ to $ABC \mid BaB \mid a$
- $A \rightarrow aA | BaC | aaaA$ to $aA \mid BaC \mid aaa$
- $B \rightarrow bBb | a | \epsilon B$ to $bBb \mid a \mid \epsilon$
- $C \rightarrow CA | ACC$ to $CA \mid AC$
- $D \rightarrow \epsilon D$ to ϵ

Step 3: Eliminate Useless Symbols

A useless symbol is a symbol that doesn't contribute to generating any terminal string. To identify and eliminate useless symbols, we follow two steps:

1. Identify non-generating symbols: A symbol is generating if it can eventually produce a string of terminal symbols.
2. Identify reachable symbols: A symbol is reachable if it can be reached from the start symbol S .

3.1. Identify non-generating symbols

We first check which non-terminals can generate terminal strings:

- $D \rightarrow \epsilon D$ to ϵ , so D is generating.
- $B \rightarrow bBb | a | \epsilon B$ to $bBb \mid a \mid \epsilon$, so B is generating because it can produce aa , or $bBbbBb$, or ϵ .
- $A \rightarrow aA | BaC | aaaA$ to $aA \mid BaC \mid aaa$, so A is generating because $aaaaaa$ is a terminal string.
- $C \rightarrow CA | ACC$ to $CA \mid AC$, but C does not generate terminal strings directly because it only generates other non-terminals A and C , which can only generate terminal strings indirectly.

So, C is not generating and should be removed.

3.2. Identify reachable symbols

Now, let's check which symbols are reachable from the start symbol S :

- $S \rightarrow ABCS \rightarrow ABC$, so SS reaches AA , BB , and CC .
- $A \rightarrow aA|BaC|aaaA \rightarrow aA \mid BaC \mid aaa$, so AA is reachable.
- $B \rightarrow bBb|a| \epsilon \rightarrow bBb \mid a \mid \epsilon$, so BB is reachable.
- $C \rightarrow CA|ACC \rightarrow CA \mid AC$, so CC is reachable from AA .
- $D \rightarrow \epsilon D \rightarrow \epsilon$, so DD is reachable.

Since CC is not generating, it should be eliminated from the grammar.

After eliminating useless symbols, the grammar becomes:

- $S \rightarrow ABC|BaB|aS \rightarrow ABC \mid BaB \mid a$
- $A \rightarrow aA|aaa|BaCA \rightarrow aA \mid aaa \mid BaC$ (remove CC from productions)
- $B \rightarrow bBb|a| \epsilon \rightarrow bBb \mid a \mid \epsilon$

Final Grammar After All Reductions:

- $S \rightarrow ABC|BaB|aS \rightarrow ABC \mid BaB \mid a$
- $A \rightarrow aA|aaa|BaA \rightarrow aA \mid aaa \mid Ba$
- $B \rightarrow bBb|a| \epsilon \rightarrow bBb \mid a \mid \epsilon$

ANS 8. C)

Let's prove that the following two languages are not context-free using the pumping lemma for context-free languages:

1. $L_1 = \{a^i \mid i \text{ is prime}\}$
2. $L_2 = \{a^{n^2} \mid n \geq 1\}$

1. Proving $L_1 = \{a^i \mid i \text{ is prime}\}$ is not context-free

We will use the pumping lemma for context-free languages to prove that L_1 is not context-free.

Pumping Lemma for CFLs:

The pumping lemma for context-free languages states that if a language L is context-free, then there exists a pumping length p such that any string w in L with length $|w| \geq p$ can be decomposed into five parts: $w = uvxyz$, satisfying the following conditions:

1. $|vxy| \leq p \mid vxy| \leq p$
2. $|vy| \geq 1 \mid vy| \geq 1$
3. For all $i \geq 0 \mid i \geq 0$, the string $uv^ix^iy^iz$ is in L .

Proof:

Let's assume, for the sake of contradiction, that $L_1 = \{a^i \mid i \text{ is prime}\}$ is context-free. By the pumping lemma, there exists a pumping length p such that any string $w \in L_1$ with $|w| \geq p$ can be decomposed into $w = uvxyz$, satisfying the above conditions.

Now, let's choose the string $w = a^q$ where q is a prime number greater than or equal to p . Thus, $w \in L_1$ with $|w| \geq p$.

By the pumping lemma, we can write $w = uvxyz$, where:

- $|vxy| \leq p \mid vxy| \leq p$
- $|vy| \geq 1 \mid vy| \geq 1$
- $uv^ix^iy^iz \in L_1$ for all $i \geq 0 \mid i \geq 0$

Key Observation:

- The string $w = a^q$ is a string of length q , where q is prime.
- Pumping v and y (which consist only of a 's) results in strings of the form a^{q+ka} for some integer k , where k is the number of additional a 's added by pumping.

If we pump v and y , we get strings of the form a^{q+ka} . For these strings to be in L_1 , the length of the string must be prime. However, if we pump the string to any value a^{q+ka} , we get a length that is not prime for most values of k . The prime property is lost after pumping, which contradicts the fact that L_1 only contains strings of prime length.

Thus, we cannot ensure that the resulting string remains in L_1 , which implies that L_1 is not context-free.

2. Proving $L_2 = \{a^{n^2} \mid n \geq 1\}$ is not context-free

Now, we will prove that $L_2 = \{a^{n^2} \mid n \geq 1\}$ is not a context-free language.

We will also use the pumping lemma for context-free languages to prove this.

Proof:

Let's assume, for the sake of contradiction, that $L_2 = \{a^{n^2} \mid n \geq 1\}$ is context-free. By the pumping lemma, there exists a pumping length p such that any string $w \in L_2$ with $|w| \geq p$ can be decomposed into $w = uvxyz$, where:

- $|vxy| \leq p$
- $|vy| \geq 1$
- $uv^i x y^i z \in L_2$ for all $i \geq 0$

Now, let's consider the string $w = a^{p^2}$, which is in L_2 since p^2 is a perfect square.

According to the pumping lemma, we can decompose $w = uvxyz$ such that the conditions of the pumping lemma hold. Let's analyze the behavior when we pump the middle part vy .

Key Observation:

- The string $w = a^{p^2}$ is of length p^2 , a perfect square.
- When we pump the middle part vy , we get strings of the form $uv^i x y^i z$, where the new length of the string is $p^2 + k$ for some integer k , corresponding to the number of a 's added by pumping.

Now, the length of the string after pumping must still be a perfect square, but after pumping, we are unlikely to get a perfect square. This is because adding or removing a number of a 's will typically break the perfect square property, and the resulting length will not be a perfect square.

For example, if $i = 2$, the new length will be $p^2 + 2k$, which is generally not a perfect square. Hence, after pumping, the resulting string will not belong to L_2 , because it will not have a length of the form n^2 for any integer n .

Thus, L_2 is not context-free because we cannot guarantee that pumping will result in strings whose length is a perfect square.

Conclusion:

Both languages $L_1 = \{a^i \mid i \text{ is prime}\}$ and $L_2 = \{a^{n^2} \mid n \geq 1\}$ are not context-free. We proved this using the pumping lemma for context-free languages, showing that pumping leads to strings that no longer satisfy the conditions for being in the respective languages.

MODULE 5

ANS 9. A)

Definition of a Turing Machine

A Turing Machine (TM) is a mathematical model of computation that defines a machine capable of simulating any algorithm. It consists of an infinite tape, a head that can read and write symbols, and a set of states that determine its operations. A Turing Machine is a formal model of computation that is widely used to study computability and complexity.

A Turing Machine consists of the following components:

1. Tape:

- An infinite sequence of cells, each containing a symbol from a finite alphabet Σ (including a special blank symbol $\#$).
- The tape is used for both input and output, and it moves left and right under the control of the machine.

2. Tape Head:

- A device that reads and writes symbols on the tape.
- It can move left, right, or stay in the current position depending on the machine's transition function.

3. State Register:

- The machine has a finite set of states Q (including a special start state and halting states).
- At any given time, the Turing Machine is in one of these states.

4. Transition Function:

- The transition function δ defines the behavior of the Turing Machine.
- It is a function: $\delta: Q \times \Sigma \rightarrow Q \times \Sigma \times \{L, R, S\}$. This means that for a given state and a symbol on the tape, the machine will:
 - Change to a new state,
 - Write a new symbol on the tape,
 - Move the tape head either left (L), right (R), or stay in the same position (S).

5. Start State:

- The state from which the machine begins its operation, denoted as q_0 .

6. Accepting and Rejecting States:

- Some states are designated as accepting (halting) states, where the machine halts and indicates success.
- Some states are rejecting (halting) states, where the machine halts and indicates failure.

Formal Definition of a Turing Machine:

A Turing Machine is formally defined as a 7-tuple:

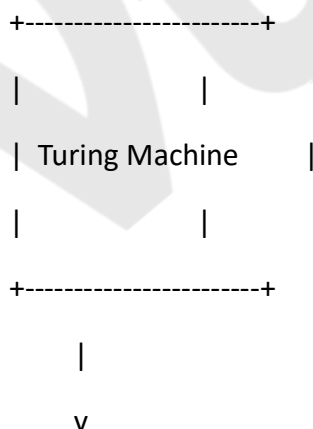
$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

Where:

- Q is the finite set of states,
- Σ is the input alphabet (excluding the blank symbol),
- Γ is the tape alphabet (including the blank symbol),
- δ is the transition function,
- q_0 is the initial state,
- q_{accept} is the accepting state,
- q_{reject} is the rejecting state.

Diagram of a Turing Machine

Here is a simple diagram to illustrate the components of a Turing Machine:



+-----+
| Tape | <----- (Infinite tape containing symbols)

| |

+-----+

|

v

+-----+

| Tape Head | <--- (Moves left, right, or stays)

+-----+

|

v

+-----+

| State Register | <--- (Holds the current state)

+-----+

|

v

+-----+

| Transition | <--- (Defines the next action based on state)

| Function |

+-----+

|

v

+-----+

| Start State | <--- (Initial state where machine begins)

+-----+

|

v

+-----+

| Accept/Reject | <--- (Accepting or rejecting state, halting)

+-----+

Working of a Turing Machine:

1. **Start State:** The Turing Machine starts in the initial state q_0 and begins reading the input from the leftmost cell of the tape.
2. **Transition Function:** Based on the current state and the symbol read by the tape head, the transition function determines:
 - The new state to transition to.
 - The symbol to write on the current tape cell.
 - The direction to move the tape head (left, right, or stay).
3. **Accepting and Rejecting:**
 - The Turing Machine continues processing until it reaches an accepting state q_{accept} (indicating successful computation) or a rejecting state q_{reject} (indicating failure or halting).

Example of Turing Machine Operation:

Consider a Turing Machine that accepts strings consisting of an even number of 1's, i.e., $L = \{0^*1^* \mid \text{number of 1's is even}\}$. The machine can proceed by:

1. Reading the first '1', changing it to a special marker (say 'X'), and moving to the right.
2. Looking for another '1', changing it to 'X' and moving back to the left.
3. Repeating this process until no more unmarked '1's are found.
4. If an even number of '1's are marked, the machine enters an accepting state; if not, it enters a rejecting state.

ANS 9. B)

Turing Machine for Language $L = \{a^n b^n c^n \mid n \geq 1\}$

We will design a Turing Machine (TM) to accept the language L , which consists of strings in the form $a^n b^n c^n$ where $n \geq 1$. This language has strings where:

- The number of 'a's is equal to the number of 'b's and the number of 'b's is equal to the number of 'c's.

Steps for Designing the Turing Machine:

The Turing Machine will:

1. Check for the first 'a', replace it with an 'X', and move the head right.
2. Move right across the 'b's, and replace the first 'b' with 'Y'.
3. Move right across the 'c's, and replace the first 'c' with 'Z'.
4. Return to the leftmost unmarked 'a' to repeat the process.
5. If the machine encounters any of the following conditions, it rejects the string:
 - If the first 'a' is not found when expected.
 - If the first 'b' is not found when expected.
 - If the first 'c' is not found when expected.
 - If any character appears out of order or if the number of 'a's, 'b's, and 'c's do not match.

The machine will halt and accept when all 'a's, 'b's, and 'c's have been marked correctly and the string is consumed.

Formal Description of the Turing Machine:

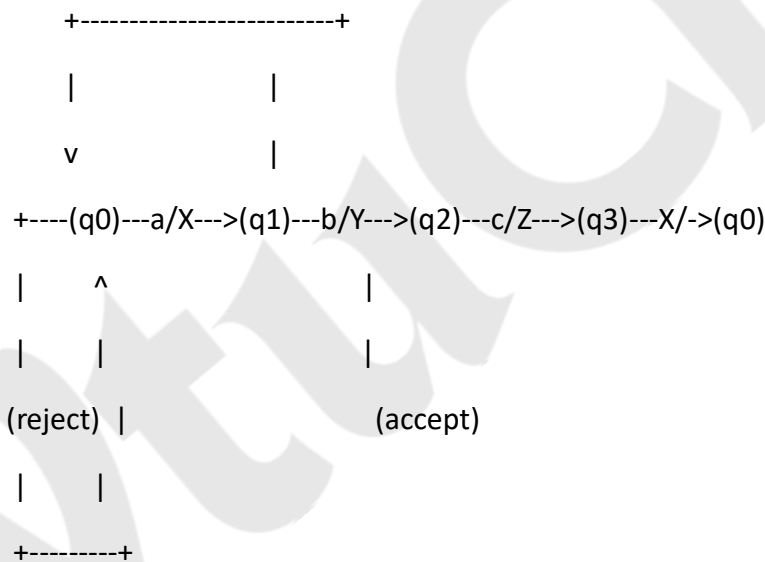
The Turing Machine will have the following states and transitions:

1. State q_0 : Start state. The machine looks for the first 'a', replaces it with 'X', and moves right.
 - q_0 , reading 'a' → write 'X', move right, transition to state q_1 .
 - q_0 , reading 'Y' or 'Z' → move right, stay in q_0 (skips over the marked characters).
2. State q_1 : The machine looks for the first 'b', replaces it with 'Y', and moves right.
 - q_1 , reading 'b' → write 'Y', move right, transition to state q_2 .
 - q_1 , reading 'X' → move right, stay in q_1 (skip over the 'a' portion).
3. State q_2 : The machine looks for the first 'c', replaces it with 'Z', and moves left.

- q_2 , reading 'c' → write 'Z', move left, transition to state q_3 .
 - q_2 , reading 'Y' → move right, stay in q_2 (skip over the marked 'b' portion).
4. State q_3 : The machine moves left across the tape, skipping over 'Y's and 'Z's, to return to the first unmarked 'a'.
- q_3 , reading 'X' → move right, transition to state q_0 (return to the unmarked 'a' to repeat the process).
 - q_3 , reading 'Y' or 'Z' → move left, stay in q_3 .
5. State q_{accept} : If the machine reaches the end of the tape and all symbols are marked, the machine enters the accepting state.
6. State q_{reject} : If an invalid situation occurs (e.g., the machine expects a 'b' but finds an 'a', or vice versa), it enters the rejecting state.

Transition Diagram

Below is the transition diagram for the Turing Machine that recognizes $L = \{a^n b^n c^n \mid n \geq 1\}$.



Explanation of the Diagram:

- State q_0 : The machine begins by looking for the first 'a'. If it finds it, it replaces it with 'X' and moves to state q_1 . If it encounters a 'Y' or 'Z', it keeps moving right until it reaches the next unmarked 'a'.

- State q1q_1: In this state, the machine looks for the first 'b'. If it finds it, it replaces it with 'Y' and moves to state q2q_2. If it encounters an 'X', it moves right and continues scanning for the next 'b'.
- State q2q_2: In this state, the machine looks for the first 'c'. If it finds it, it replaces it with 'Z' and moves left to return to the previous unmarked 'a' by transitioning to state q3q_3.
- State q3q_3: The machine moves left, skipping over any 'Y' or 'Z' encountered, and goes back to the leftmost unmarked 'a'. It then transitions to q0q_0 to repeat the process.
- Accepting State: The machine will enter the accepting state if all the 'a's, 'b's, and 'c's are replaced by 'X', 'Y', and 'Z', respectively, and there are no more characters to process.
- Rejecting State: If the machine encounters an unexpected situation (e.g., trying to find a 'b' when there are no more 'a's or trying to find a 'c' when there are no more 'b's), it will reject the string.

Example: String aabbccaabbcc

Let's simulate the transitions for the input string "aabbcc".

Initial Configuration:

Tape: a a b b c c Head: Points to the first 'a'

1. Start (q0): Read 'a', replace it with 'X', move right.
 - Tape: X a b b c c
 - Head: Points to 'a' (second position)
2. State (q1): Read 'a', move right (skip to 'b').
 - Tape: X a b b c c
 - Head: Points to 'b' (third position)
3. State (q1): Read 'b', replace it with 'Y', move right.
 - Tape: X Y b b c c
 - Head: Points to 'b' (fourth position)
4. State (q2): Read 'b', move right (skip to 'c').
 - Tape: X Y b b c c

- Head: Points to 'c' (fifth position)
- 5. State (q2): Read 'c', replace it with 'Z', move left.
 - Tape: X Y b b Z c
 - Head: Points to 'b' (fourth position)
- 6. State (q3): Read 'b', move left (skip over 'Y').
 - Tape: X Y b b Z c
 - Head: Points to 'Y' (third position)
- 7. State (q3): Read 'Y', move left.
 - Tape: X Y b b Z c
 - Head: Points to 'X' (second position)
- 8. State (q0): Read 'X', move right (return to 'a').
 - Tape: X Y b b Z c
 - Head: Points to 'b'
- 9. State (q1): Read 'b', move right (skip to 'c').
 - Tape: X Y b b Z c
 - Head: Points to 'c'
- 10. State (q2): Read 'c', move right to accept the input since all symbols are processed.

ANS 10. A)

Turing Machine to Accept Palindromes Over $\{a,b\}^*$

A palindrome is a string that reads the same forward and backward. We will design a Turing Machine that accepts strings which are palindromes over the alphabet $\{a,b\}^*$.

Approach:

1. The Turing Machine starts by scanning the leftmost character of the string.
2. It replaces the first unmarked character (either 'a' or 'b') with a special marker (let's use 'X' for 'a' and 'Y' for 'b'), then moves to the right to find the corresponding matching character from the right end of the string.

3. If a matching character is found, it replaces that character with the marker ('X' or 'Y'), then moves back to the leftmost unmarked character and repeats the process.
4. If it ever finds a mismatch (e.g., the first unmarked character does not match the last unmarked character), it rejects the string.
5. The machine accepts the string if all characters are successfully paired and marked, and the tape head reaches the blank space.

Formal Description:

Let the Turing Machine MM for accepting palindromes over $\{a,b\}$ be defined as follows:

- States: $Q = \{q_0, q_1, q_2, q_3, q_4, q_{\text{accept}}, q_{\text{reject}}\}$
- Alphabet: $\Sigma = \{a, b\}$
- Tape Alphabet: $\Gamma = \{a, b, X, Y, \#\}$ (including 'X' for 'a', 'Y' for 'b', and the blank symbol '#').
- Start State: q_0
- Accepting State: q_{accept}
- Rejecting State: q_{reject}

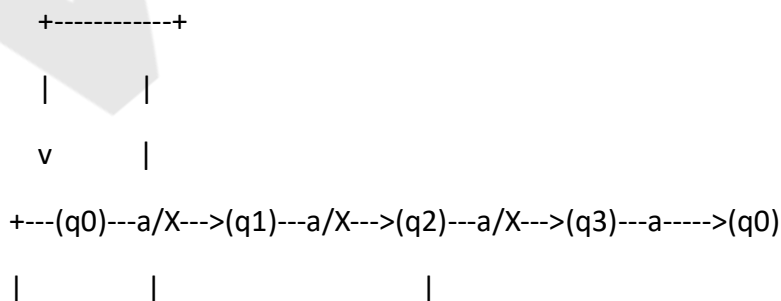
Transition Function:

1. State q_0 (Looking for the leftmost unmarked symbol):
 - $q_0, a \rightarrow X, R, q_1$ (If 'a', mark with 'X', move right, and transition to q_1).
 - $q_0, b \rightarrow Y, R, q_1$ (If 'b', mark with 'Y', move right, and transition to q_1).
 - $q_0, \# \rightarrow \#, S, q_{\text{accept}}$ (If the tape is empty, accept).
2. State q_1 (Moving right to find the matching character):
 - $q_1, a \rightarrow a, R, q_1$ (Move right, skip over unmarked 'a').
 - $q_1, b \rightarrow b, R, q_1$ (Move right, skip over unmarked 'b').
 - $q_1, X \rightarrow X, R, q_2$ (Found an 'X', move right to find corresponding character).

- $q_1, Y \rightarrow Y, R, q_2$ (Found a 'Y', move right to find corresponding character).
3. State q_2 (Looking for the rightmost unmarked symbol):
- $q_2, a \rightarrow X, L, q_3$ (If 'a' is found, mark with 'X' and move left).
 - $q_2, b \rightarrow Y, L, q_3$ (If 'b' is found, mark with 'Y' and move left).
 - $q_2, \# \rightarrow \#, S, q_{\text{reject}}$ (If no matching symbol found, reject).
4. State q_3 (Moving left to find the leftmost unmarked symbol):
- $q_3, a \rightarrow a, L, q_3$ (Move left, skip over unmarked 'a').
 - $q_3, b \rightarrow b, L, q_3$ (Move left, skip over unmarked 'b').
 - $q_3, X \rightarrow X, L, q_0$ (If 'X' is found, go back to q_0 to repeat the process).
 - $q_3, Y \rightarrow Y, L, q_0$ (If 'Y' is found, go back to q_0 to repeat the process).
5. Final Acceptance (State q_{accept}):
- If all symbols are marked and the machine reaches the blank symbol $\#$, it enters the accepting state.
6. Rejection (State q_{reject}):
- The machine enters the rejecting state if it finds a mismatch or if an unexpected situation occurs (e.g., reaching a blank symbol when not expected).

Transition Diagram

Below is the transition diagram for the Turing Machine that recognizes palindromes over $\{a, b\}$.



	b/Y--->(q1)	b/Y
	Y/Y--->(q2)	Y/Y
	(Accept) <---	
	v	
+----- (q_reject) --+		
Reject		

How the Turing Machine Works:

Let's simulate the Turing Machine for an example string: abba.

1. Start State (q0):
 - The machine reads the first 'a', replaces it with 'X', and moves right to q1q_1.
2. State (q1):
 - The machine moves right and finds 'b', then moves to q2q_2.
3. State (q2):
 - The machine reads the last 'b', replaces it with 'Y', and moves left to q3q_3.
4. State (q3):
 - The machine moves left and finds the 'X' (previously marked 'a'). It returns to q0q_0.
5. State (q0):
 - The machine reads the 'X' (the first marked 'a') and moves right to the next unmarked character, which is 'Y'.
6. State (q1):
 - The machine checks if it has reached the blank symbol and accepts the string.

This process works similarly for all palindromes, and the Turing Machine will accept the input if it is a palindrome and reject it if it is not.

ANS 10. B)

Recursively Enumerable Language (RE Language)

A recursively enumerable language (RE) is a type of formal language that can be recognized by a Turing machine. It is also known as a Turing-recognizable language. These languages are the ones for which there exists a Turing machine that can accept any valid string in the language and either halt or run indefinitely when the string is not in the language. In other words, a language is recursively enumerable if there exists a Turing machine that will:

- Accept all strings that belong to the language (halt in the accepting state).
- Reject or never halt for strings that do not belong to the language.

Formal Definition:

A language L is recursively enumerable if there exists a Turing machine M such that:

- For any string $w \in L$, M halts and accepts w .
- For any string $w \notin L$, M either halts and rejects w or runs forever (does not halt).

Key Points about Recursively Enumerable Languages:

1. Decidability vs Recognizability:

- A decidable language is one for which there exists a Turing machine that halts for every input and correctly decides whether the input is in the language or not (i.e., it always halts with either "accept" or "reject").
- A recursively enumerable language is one for which there exists a Turing machine that accepts strings in the language, but for strings not in the language, the machine may run forever (i.e., it might not halt).

2. Examples:

- The set of all valid C programs is a recursively enumerable language because a Turing machine can recognize whether a string is a valid C program, but it may never halt if the program contains an infinite loop.
- The Halting Problem itself is a classic example of a problem that is RE but not decidable. The language of Turing machines that halt on a given input is recursively enumerable because we can simulate the Turing machine and see if it halts. If it halts, we accept; if it doesn't, we may never halt.

3. Properties of RE Languages:

- Closure Properties: The class of recursively enumerable languages is closed under union, but not under intersection or complementation. That is, while the union of two RE languages is also RE, the intersection or complement of two RE languages might not be RE.
- Non-Closure under Complement: The complement of a recursively enumerable language might not be recursively enumerable. This is a key distinction from decidable languages, which are closed under complementation.

Multi-Tape Turing Machine

A multi-tape Turing machine is an extension of the standard (single-tape) Turing machine. In this model, there are multiple tapes (instead of just one), and each tape has its own read/write head. The machine operates in much the same way as a single-tape Turing machine, except now it can access multiple tapes simultaneously, each one operating independently of the others. The machine is still governed by a finite set of states, and it has a transition function that dictates how the head of each tape moves based on the current state and the symbols read from the tapes.

Components of a Multi-Tape Turing Machine:

1. Multiple Tapes: A multi-tape Turing machine has more than one tape. Each tape is infinite in length and serves as the storage for the machine.
2. Multiple Tape Heads: Each tape has its own tape head that can read, write, and move independently.
3. Transition Function: The transition function in a multi-tape Turing machine is slightly more complex than that of a single-tape machine because it takes the symbols read from all tapes and uses them to decide the next state, the symbols to write on each tape, and the direction in which to move each tape head.

Formal Definition:

A multi-tape Turing machine is defined as a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where:

- Q is the finite set of states.
- Σ is the input alphabet.
- Γ is the tape alphabet (which includes symbols that can be written on the tape).
- δ is the transition function that now operates on multiple tapes.

- q_0 is the start state.
- q_{accept} is the accepting state.
- q_{reject} is the rejecting state.

Key Features of Multi-Tape Turing Machines:

1. More Efficient Computation:
 - Multi-tape Turing machines can often perform computations more efficiently than single-tape machines. For example, they can simulate certain operations, like copying data, more quickly by writing to one tape and reading from another.
2. Computational Power:
 - A multi-tape Turing machine is more powerful than a single-tape machine in terms of time complexity, but both have the same computational power in terms of the class of languages they can recognize. Any language that can be recognized by a multi-tape Turing machine can also be recognized by a single-tape Turing machine (though potentially with a different time complexity). This is due to the Church-Turing thesis which states that the single-tape Turing machine is as powerful as any computational model, including multi-tape Turing machines.
3. Simulating Multi-Tape Turing Machine on a Single-Tape Machine:
 - While multi-tape Turing machines may be more efficient, a single-tape Turing machine can still simulate a multi-tape machine. However, the simulation may take more time. For instance, a multi-tape Turing machine that works in $O(n)O(n)$ time can be simulated by a single-tape Turing machine in $O(n^2)O(n^2)$ time.

Example:

Consider a multi-tape Turing machine that copies the input string to a second tape:

- Tape 1: Contains the input string.
- Tape 2: Is initially empty and will hold a copy of the input string.

The multi-tape Turing machine works by scanning the input string on Tape 1, reading each symbol, and copying it to Tape 2. The heads on both tapes move simultaneously, and the machine halts once the entire string has been copied.

A single-tape Turing machine would require additional steps to simulate this process, such as moving back and forth between tapes and using additional markers.

Summary:

1. Recursively Enumerable Languages (RE):

- RE languages are those for which a Turing machine can recognize and accept strings that belong to the language.
- For strings not in the language, the machine may run forever without halting.
- These languages are not necessarily decidable because the machine may not halt on all inputs.

2. Multi-Tape Turing Machines:

- A multi-tape Turing machine extends the concept of the standard Turing machine by having multiple tapes and heads.
- These machines are more efficient in certain tasks but do not increase the computational power of Turing machines.
- A multi-tape machine can be simulated by a single-tape machine, though with a potential increase in time complexity.